

What I delivered

A concise record of the implemented system and the choices behind it.

CORE RESULT

The dispatcher and officer clients now share one Supabase source of truth, with push delivery and live snapshots around a database-enforced incident lifecycle.

END-TO-END WORKFLOW

1

Log

Dispatcher records caller, location, type, priority and description.

2

Dispatch

Supabase validates that an eligible officer exists, then broadcasts the incident.

3

Notify

The Edge Function sends an Expo push only to registered on-duty devices.

4

Claim

One transactional RPC chooses the first officer; other clients reconcile live.

5

Respond

The owner moves Claimed to En route to On scene in the mobile app.

6

Close

One final report resolves the incident and appears in the dispatcher archive.

IMPLEMENTATION MAP

WEB

Next.js 16

Protected dispatcher routes, intake, live queue, roster, response detail and report archive.

MOBILE

Expo SDK 57

Secure officer sessions, duty state, queue, claim, response timeline, report and push registration.

BACKEND

Supabase

Postgres, Auth, RLS, transactional RPCs, private Realtime topics and an Edge Function.

Quality evidence: TypeScript, Expo lint, Expo config validation and Android production bundling passed; the live push and realtime flow was manually exercised.

Trade-offs and next steps

Shortcuts were chosen to protect the required core loop and deadline.

1

Seeded accounts

No public registration. This matches the brief and keeps role assignment controlled through Supabase app metadata.

2

Database over client state

Claims and status changes use narrow RPCs and constraints. This adds SQL work but prevents two clients from creating conflicting truth.

3

Private invalidations

Realtime messages contain IDs, then clients re-fetch through RLS. This is safer than broadcasting caller or report details.

4

Text-only reports

Photos were intentionally left out. The required response summary, outcome and server resolution time were prioritized.

5

Android preview first

Expo/EAS provides the installable assessment build. Free-tier cloud queues may delay a fresh APK but do not stop the submitted cloud job.

KNOWN LIMITATION

A sudden app kill can leave duty state stale because presence has no heartbeat/TTL yet.

SUBMISSION REMINDER

The final single PDF still needs the live URL, APK link, credentials and demo-video link.

Next: add presence expiry, delivery receipts, automated end-to-end tests, monitoring and optional incident photos.